



Universidad Autónoma de Nuevo León
Facultad de Ingeniería Mecánica y Eléctrica

Artificial Intelligence

Assignment 5.- Artificial Neural Network: feedforward

Ing. Daniel Isaías López Páez

Ximena Villarreal Dávila	2177714
Benito Woo Zozaya	2177866
David Palma Cabañez	2177805
Carlos Arturo Arguelles Segura	2096281
LMV N1	Group: 001

San Nicolas de los Garza, N.L., 22 de Mayo de 2026

Introduction to the topic

Artificial Neural Networks (ANNs) are computational models inspired by the structure and function of biological neural networks in the human brain. Composed of layers of interconnected nodes (neurons), ANNs are capable of learning complex patterns from data through a process of adjusting connection weights during training.

ANNs have become a cornerstone of modern machine learning, enabling breakthroughs in image recognition, natural language processing, speech synthesis, and many other domains. Their ability to approximate non-linear functions makes them particularly powerful for tasks where hand-crafted rules are insufficient.

A Feedforward Neural Network (FFNN) is the simplest type of ANN. Information flows in one direction — from the input layer through one or more hidden layers to the output layer — with no cycles or feedback connections. Each neuron applies a weighted sum of its inputs followed by a non-linear activation function.

The MNIST (Modified National Institute of Standards and Technology) dataset is a widely used benchmark for handwritten digit recognition. It consists of 70,000 grayscale images of handwritten digits (0–9), each 28×28 pixels, split into 60,000 training samples and 10,000 test samples.

Methodology

- 1) The model was implemented in Python using the following libraries:

```
import numpy as np
import matplotlib.pyplot as plt
import keras
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.utils import to_categorical
```

- 2) The dataset was loaded using the explicit Keras API call:

```
keras.datasets.mnist.load_data(path="mnist.npz")
```

This returns the default 60,000 / 10,000 train–test split. Pixel values were normalized to the range [0, 1] by dividing by 255. Labels were one-hot encoded into 10-class vectors using `to_categorical()`.

- 3) The Feedforward Neural Network was designed with the following structure:

```
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax'),
])
```

- 4) The model was compiled with the following settings:

```
model.summary()

model.compile(
    loss      = 'categorical_crossentropy',
    optimizer = 'adam',
    metrics   = ['accuracy'],
)
```

The model was trained using `model.fit()` for 15 epochs with a batch size of 64. A 10% validation split was applied to monitor overfitting during training.

```
history = model.fit(
    train_data,
    train_labels_ohe,
    epochs      = 15,
    batch_size  = 64,
    validation_split= 0.1,
    verbose     = 1,
)
```

- 5) Model performance was assessed on the test set using `model.evaluate()`. Subsequently, 5 sample predictions were generated using `model.predict()` and compared against the actual labels, displaying both values for each sample.

```
test_loss, test_accuracy = model.evaluate(test_data, test_labels_ohe, verbose=0)
print(f"\nTest Loss      : {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

```

print("\n--- 5 Sample Predictions ---")
predictions = model.predict(test_data[:5])

for i in range(5):
    predicted_digit = np.argmax(predictions[i])
    actual_digit = test_label[i]
    correct = "✓" if predicted_digit == actual_digit else "X"
    print(f" Sample {i+1}: Prediction = {predicted_digit} | Actual = {actual_digit} {correct}")

```

Results

After training for 15 epochs, the model achieved strong performance on the MNIST test set. The use of three hidden ReLU layers followed by a softmax output layer proved effective for classifying the 10 digit categories.

The `model.evaluate()` function was used to report performance on the 10,000 test samples:

```

Test Loss      : 0.0976
Test Accuracy: 97.67%

```

These results are consistent with expected performance for a well-tuned dense network on MNIST, demonstrating that the selected architecture, loss function, and optimizer are appropriate for this task.

Five samples from the test dataset were selected, and the model's predictions were compared against their actual labels:

```

--- 5 Sample Predictions ---
1/1 ----- 0s 83ms/step
Sample 1: Prediction = 7 | Actual = 7 ✓
Sample 2: Prediction = 2 | Actual = 2 ✓
Sample 3: Prediction = 1 | Actual = 1 ✓
Sample 4: Prediction = 0 | Actual = 0 ✓
Sample 5: Prediction = 4 | Actual = 4 ✓

```

All five selected samples were correctly classified.

General Conclusion

This assignment demonstrated the complete workflow of designing, implementing, and evaluating a Feedforward Neural Network using the MNIST handwritten digit dataset. By leveraging Keras's high-level API, the model was built efficiently with minimal code while achieving competitive accuracy (~98%).

Key takeaways include the importance of data normalization for convergence, the effectiveness of ReLU activations in hidden layers, and the suitability of softmax for multi-class classification. The Adam optimizer proved reliable for this task, adapting the learning rate dynamically throughout training.

Future improvements could include adding Dropout layers to reduce overfitting, increasing the number of epochs, or experimenting with Convolutional Neural Networks (CNNs) which are architecturally better suited for image classification tasks.

REPOSITORY LINK:

https://github.com/xvillarreal-d-max/ARTIFICIAL_INTELLIGENCE_TEAMXBDC/blob/ff30b619aababc044b0b2ac25294a763bca09075/Assignment_5.ipynb

Bibliographic Reference:

MNIST digits classification dataset. (s/f). Keras.io. Recuperado el 23 de mayo de 2026, de <https://keras.io/api/datasets/mnist/>